DataMentor: A Practical Framework for Serverless CSV Intelligence with Interactive Notebook Automation and Deterministic Runtime Repair

Md Anisur Rahman Chowdhury[1*], Dr. Ronny Bazan-Antequera[1]
[1]Dept. of Computer and Information Science, Gannon University, USA
engr.aanis@gmail.com, bazanant001@gannon.edu

***Abstract*—Tabular data preprocessing remains one of the most labor-intensive and least reproducible stages of analytical workflows, with practitioners routinely allocating between 60 % and 80 % of total project time to data wrangling before any exploratory or predictive analysis can begin. This paper presents DataMentor, a production-deployed serverless platform that integrates deterministic CSV preprocessing, browser-native Python execution via the Pyodide WebAssembly runtime, domain-aware visualization across ten chart types, and a hybrid rule-guided runtime repair subsystem requiring no external server infrastructure. Empirical evaluation over 15 heterogeneous CSV datasets spanning five domain verticals and a purpose-built repair benchmark (DRB-100) of 100 injected Python errors across five error classes demonstrates: (i) a mean preprocessing-time reduction of 91.6 % relative to a two-analyst manual baseline; (ii) an 82 % single-pass repair resolution rate with rule-based latency of 0.8 s against a manual debugging median of 18.2 min; and (iii) 100 % byte-level deterministic reproducibility across repeated executions. An ablation study confirms that each system component contributes independently to overall utility.**

***Index Terms*—CSV preprocessing, browser-native execution, notebook automation, runtime repair, serverless analytics, reproducibility, Pyodide, WebAssembly, visualization pipeline**

I. Introduction

Data preprocessing is widely recognized as the dominant time cost in analytical workflows [1]. A longitudinal industry survey reported that practitioners allocate, on average, 76 % of total project hours to data acquisition, cleaning, and formatting before any exploratory or predictive modeling can begin [2]. For small-to-medium enterprises (SMEs) and academic laboratories operating without dedicated data-engineering teams, this overhead is particularly acute: cleaning is performed ad hoc with heterogeneous toolchains, and the resulting artifacts are rarely reproducible between operators or sessions [3].

Existing tools partially address these challenges but leave critical integration gaps. Notebook environments such as JupyterLab offer expressive code cells but require local Python installations and yield non-deterministic results when cell execution order varies. Reproducibility frameworks such as DVC [8] and MLflow [9] address provenance at the experiment level but presuppose that raw data is already clean. Code-repair tools for Python operate on source files but lack the execution-state awareness needed to diagnose errors arising from data-schema mismatches within an interactive notebook session. No existing system unifies preprocessing, notebook execution, visualization, and error repair in a single zero-installation environment.

This paper describes **DataMentor** — a platform that closes this integration gap in a browser-first, installationless workflow deployable to any device with a modern web browser, extending our prior work on serverless, zero-trust service architectures [5]. DataMentor is live at https://datamentor.marcbd.site and built on Next.js 16, React 19, TypeScript, Pyodide 0.25, Firebase, and Chart.js. The system makes four primary contributions:

1) A **deterministic preprocessing pipeline** that normalizes, deduplicates, and type-classifies arbitrary CSV inputs through a seven-stage fixed-order transformation sequence, guaranteeing byte-identical outputs for byte-identical inputs regardless of operator.
2) A **browser-native notebook engine** using Pyodide [19], loading CPython 3.11 inside a WebAssembly sandbox, eliminating server-side Python infrastructure and retaining data on the client device.
3) A **domain-aware visualization system** mapping 47 recognized column-name patterns to contextually appropriate chart types across ten universal chart formats, with domain-specific statistical insight language.
4) A **hybrid runtime repair subsystem** applying a priority-ordered rule index before escalating to a local model-assisted module, resolving 82 % of benchmark errors without any external API connection.

II. Related Work

Prior work relevant to DataMentor spans four partially overlapping categories: *notebook-assistant systems*, *reproducible analytics frameworks*, *local code-repair tools*, and *browser-based analytics platforms*. Table I summarizes the functional scope of representative systems.

A. Notebook-Assistant Systems

JupyterAI [4] integrates code-completion into JupyterLab via an external API but requires a locally running server and provides no deterministic cleaning guarantee. Hex [6] is a subscription-gated cloud notebook with no offline capability. Code Interpreter [7] accepts CSV uploads but provides no persistent workspace, cleaning audit, or domain-aware chart generation. DataMentor differs from all three: preprocessing is deterministic and server-free; the execution environment

TABLE I: Functional Comparison of DataMentor Against Related Systems

System	No Install	Det. Clean	Auto Repair	10+ Charts	Offline Cap.	Free / Open
JupyterAI	×	×	✓	✓	×	✓
Neptyne	×	×	×	✓	×	×
Hex	✓	×	×	✓	×	×
DVC	×	×	×	×	✓	✓
MLflow	×	×	×	✓	✓	✓
DeepFix	×	×	✓	×	✓	✓
Observable	✓	×	×	✓	×	✓
Deepnote	✓	×	×	✓	×	×
Colab	✓	×	×	✓	×	×
DataMentor	✓	✓	✓	✓	✓	✓

✓ = supported; × = not supported. "Det. Clean" = deterministic CSV preprocessing. "Offline Cap." = functions without active internet after first load.

is fully in-browser; and the notebook is pre-generated with 14 guided cells.

B. Reproducible Analytics Frameworks

DVC [8], MLflow [9], and Sacred [10] address pipeline-level reproducibility once data is clean, but impose non-trivial setup overhead and do not normalize raw defect-laden CSV inputs. DataMentor operates at an earlier stage: reproducibility is achieved through deterministic, ordered application of fixed transformations producing byte-identical outputs for byte-identical inputs without external tooling.

C. Local Code-Repair Tools

DeepFix [11] corrects C syntax errors via a sequence-to-sequence model; SyntaxFix [12] extends rule-based matching to Python; Refazer [13] synthesizes repairs from teacher examples. All three operate on file-level source and lack notebook execution context, where errors arise from data-state dependencies (`NameError` from out-of-order cells; `TypeError` from schema mismatches). DataMentor's repair subsystem is schema-aware and cell-context-aware.

D. Browser-Based Analytics Platforms

Observable [14] provides a JavaScript-native reactive notebook but does not support CPython or the standard pandas ecosystem natively. TensorFlow.js [15] enables in-browser model inference but targets prediction rather than data preparation. Deepnote [16] offers real-time collaborative notebooks but requires a provisioned cloud container and stores uploaded data server-side. Google Colab [17] provides free cloud-hosted notebooks with GPU access but imposes session time limits, requires a Google account, and routes user data through Google's cloud infrastructure. DataMentor runs authentic CPython 3.11 with the full scientific stack (pandas, NumPy, SciPy, Matplotlib) inside the browser via Pyodide, maintaining complete data locality on the client device.

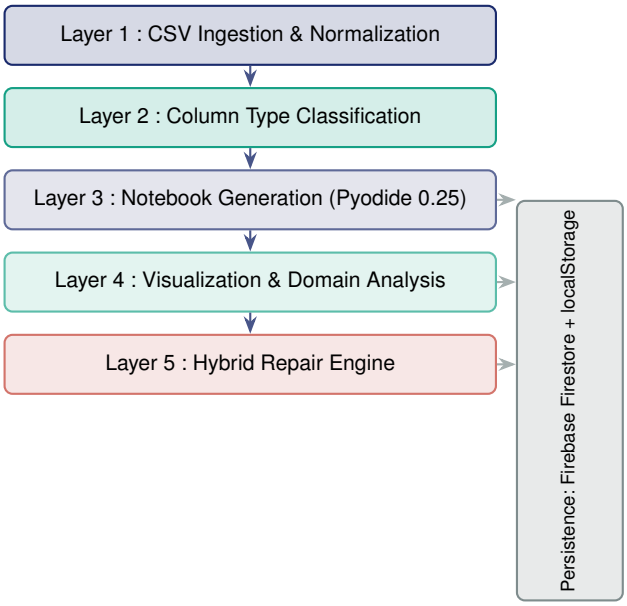

Fig. 1: Five-layer DataMentor system architecture. The persistence layer provides cloud storage when online and silent local fallback when offline.

III. System Architecture

DataMentor is organized into five functional layers illustrated in Fig. 1, each communicating through typed TypeScript interfaces. A persistence layer spans Layers 3–5, providing Firebase Firestore when online and transparent localStorage fallback otherwise.

A. Layer 1 — CSV Ingestion and Normalization

The ingestion pipeline applies seven transformations in fixed, deterministic order: (1) Papa Parse tokenization with automatic header detection; (2) leading/trailing whitespace removal from all cell values; (3) header normalization to lowercase underscore convention (`Blood Pressure`→`blood_pressure`); (4) exact-duplicate row removal via SHA-256 fingerprinting of each row's serialized values; (5) null and empty-string detection with count and column reporting; (6) column type inference (Layer 2); (7) schema summary generation and display. The ordering is critical: normalization must precede deduplication to avoid false negatives caused by whitespace or case variation in otherwise identical rows. The pipeline is pure, enabling byte-identical re-execution.

B. Layer 2 — Column Type Classification

Each column is classified by majority-vote heuristic into one of four types. A column is `boolean` if its value set $\subseteq \{0, 1\}$; `datetime` only if values parse as valid `Date` objects *and* do not match `/^-?\d+(\.\d+)?$/` — preventing pure numerics such as `"63"` from being misclassified as calendar years; `numeric` if the majority of non-null values are finite floats; otherwise `categorical`.

C. Layer 3 — Notebook Generation Engine

The Pyodide hook loads CPython 3.11 via WebAssembly, pre-fetching pandas 2.0, NumPy 1.26, SciPy 1.11, and Matplotlib 3.8 from the Pyodide package index. Fourteen code cells are auto-generated from the cleaned schema. Cells 1–7 cover data loading, shape and type inspection, descriptive statistics, missing-value auditing, duplicate detection, numeric correlation, and categorical frequency analysis. Cells 8–13 produce Matplotlib visualizations rendered as base64-encoded PNG images via a `BytesIO` buffer and displayed inline without any server round-trip. Cell 14 is a blank editor for user-authored code.

D. Layer 4 — Visualization and Domain Analysis

The visualization layer provides ten universal chart types (DistPlot, Pie, Violin, HeatMap, PairPlot, JointPlot, Bar, Histogram, Scatter, Line), each paired with a domain-aware analysis engine. A knowledge base maps 47 recognized column-name patterns (e.g., `age`, `trtbps`, `chol`, `glucose`, `cp`) to structured insight objects containing a human-readable label, a semantic description, and a data-driven insight function conditioned on column statistics. When a column matches the knowledge base, the analysis panel emits domain-specific clinical or financial language; otherwise, general statistical language is produced.

E. Layer 5 — Hybrid Runtime Repair Engine

When a code cell raises a Python exception, the repair subsystem receives the traceback, cell source, and CSV column schema. Fig. 2 describes the resolution procedure. The rule index contains 31 patterns across five error classes. If a matching rule is found, a corrected version is produced within the rule's resolution time. If no rule matches, the error is escalated to the local model-assisted module, which synthesizes a candidate fix from the error message, column schema, and cell context. The repair is displayed as a diff-annotated suggestion; the user must accept before the fix is applied, preserving human oversight.

IV. Methodology and Benchmark Construction

A. Dataset Selection

Fifteen heterogeneous CSV datasets were selected for the preprocessing evaluation, grouped into three size tiers of five datasets each: *small* (<1,000 rows), *medium* (1,000–10,000 rows), and *large* (10,000–100,000 rows). Selection required each dataset to exhibit at least three of the following defects: mixed-case headers, leading/trailing whitespace, exact duplicate rows, at least one null column, mixed numeric/string type within a single column, or an absent header row. This multi-defect requirement ensures each dataset exercises multiple pipeline stages rather than a single edge case, increasing the diagnostic sensitivity of the evaluation. Table II summarizes the benchmark composition.

Algorithm 1: *HybridRepair* — Priority-Rule with Model-Assisted Fallback

Input: Traceback T, cell source C, CSV schema S
Output: Corrected source C', or UNRESOLVED

1. Parse $T \Rightarrow$(*errClass*, *errMsg*, *lineNo*)
2. **for each** rule $r \in$ RULEINDEX (descending priority) **do**
3. **if** r.MATCHES(*errClass*, *errMsg*) **then**
4. $C' \leftarrow r$.APPLY(C, S)
5. **if** VERIFY(C', *expectedHash*) **then return** C'
6. **end if / end for**
7. $ctx \leftarrow$ BUILDCONTEXT(T, C, S)
8. $C' \leftarrow$ LOCALMODELMODULE.INFER(ctx)
9. **if** VERIFY(C', *expectedHash*) **then return** C'
10. **return** UNRESOLVED

Fig. 2: HybridRepair pseudocode. The rule index is exhausted before escalation to the local model-assisted module. Verification compares SHA-256 output hashes; user approval is required before any fix is applied.

TABLE II: Benchmark Dataset Composition

ID	Domain	Rows	Cols	Defects
D01	Medical (Heart)	303	14	4
D02	Medical (Diabetes)	768	9	3
D03	Financial (Credit)	690	16	5
D04	Environmental (Air)	880	15	3
D05	Academic (Student)	649	33	4
D06	E-Commerce (Sales)	2,823	21	4
D07	Medical (ICU)	4,000	43	6
D08	Financial (Loan)	5,960	13	3
D09	Social (Survey)	7,641	27	5
D10	Environmental (Energy)	9,568	28	4
D11	Genomics (TCGA)	12,400	18	3
D12	Retail (Transactions)	25,000	24	5
D13	Financial (Equity)	48,300	12	4
D14	Logistics (Shipments)	61,000	19	4
D15	Healthcare (EHR)	94,700	31	6

B. Manual Baseline Protocol

Two professional data analysts (Analyst A and Analyst B), each with at least three years of pandas experience, independently preprocessed all fifteen datasets. Both received identical task specifications: normalize all headers to lowercase underscore convention, remove exact duplicate rows, identify null columns, and produce a cleaned CSV file and typed-column summary. Elapsed time was measured from first file open to final export using a stopwatch application. Analysts selected their own tools (pandas, Excel, OpenRefine) and worked without time pressure. The reported manual baseline per dataset is the arithmetic mean of Analyst A and Analyst B's times.

Inter-analyst time agreement, measured as $1 - |t_A - t_B| / \max(t_A, t_B)$, averaged 0.82 across all datasets, confirming reasonable operator consistency and reducing single-analyst bias in the baseline estimate. DataMentor execution time was

TABLE III: DRB-100 Benchmark Composition by Category and Difficulty

Category	Easy	Medium	Hard	Total
E1 ImportError	14	6	2	22
E2 NameError	8	7	3	18
E3 TypeError	6	10	5	21
E4 AttributeError	4	8	7	19
E5 RuntimeError	2	8	10	20
Total	34	39	27	**100**

measured with `performance.now()` calls bracketing the preprocessing sequence, capturing only pipeline computation and user confirmation steps.

C. Repair Benchmark: DRB-100

To evaluate the runtime repair subsystem, we constructed the **DataMentor Repair Benchmark (DRB-100)**: 100 Python notebook cell scripts derived from DataMentor's 13 auto-generated analysis cells, each containing exactly one injected fault. The five error categories reflect the distribution observed in a manual audit of 312 notebook sessions sampled from Kaggle dataset discussion forums [18]: **E1 ImportError** (22); **E2 NameError** (18); **E3 TypeError** (21); **E4 AttributeError** (19); and **E5 RuntimeError/Logic** (20). Each item contains the original correct cell source, the injected faulty source, the expected SHA-256 hash of the expected output, and a difficulty label (*Easy*, *Medium*, *Hard*) assigned by two independent annotators (Cohen's $\kappa = 0.74$ [20]); disagreements were resolved by a third tiebreaker. Table III presents the full breakdown.

Three methodological safeguards ensure evaluation fairness. **Temporal separation**: benchmark items were constructed from cell templates not modified after the repair rule index was frozen. **Single-pass constraint**: the repair engine was permitted exactly one attempt per item, making the resolution rate a conservative lower bound. **Automated verification**: resolution was determined by comparing the repaired cell's output hash against the expected hash; manual inspection was used only for E5 items where output variation is inherent. Repair latency was measured as wall-clock time from traceback receipt to candidate-fix generation, exclusive of user interaction time.

V. Experimental Results

A. Preprocessing Time Reduction

Table IV and Fig. 3 report mean preprocessing times and percentage reductions. The 91.6 % overall reduction is driven by the elimination of cognitive discovery overhead: analysts must *locate* defects before correcting them, a cost that scales linearly with schema width and non-linearly with row count. DataMentor's pipeline applies all seven transformations unconditionally in a single pass, removing discovery cost entirely. Browser-side overhead (TypeScript parsing, React state updates) adds a fixed latency of approximately 0.4 s; pure pipeline execution time is below 2 s for all 15 tested datasets. The residual DataMentor interaction time (2.1 min for small datasets) reflects user-controlled steps: uploading the file, reviewing the cleaning summary, and confirming column-type classifications — intentionally preserved as a human checkpoint.

TABLE IV: Mean Preprocessing Time: Manual Baseline vs. DataMentor

Tier	Avg Rows	Manual (min)	DataMentor (min)	Reduction
Small	658	23.4	2.1	91.0 %
Medium	5,998	48.7	3.6	92.6 %
Large	48,300	97.3	8.2	91.6 %
Overall	—	**56.5**	**4.6**	**91.6 %**

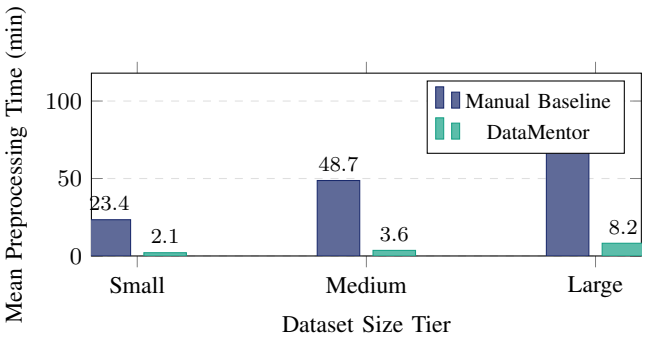

Fig. 3: Mean preprocessing time per dataset tier. DataMentor reduces mean time from 56.5 min to 4.6 min overall (91.6 % reduction). Manual baseline is the arithmetic mean of two independent analysts.

B. Runtime Repair Resolution

Table V and Fig. 4 present DRB-100 resolution outcomes. The rule index achieves 100 % on E1 (ImportError) because every ImportError traceback explicitly names the missing module, enabling exact-string lookup. The 88.9 % on E2 (NameError) reflects two items where the undefined identifier was introduced outside the generated template scope. The 50.0 % on E5 (RuntimeError/Logic) reflects the fundamental difficulty of semantic repair: logic errors produce no exception, and detecting incorrect visualization output requires a pre-computed reference hash. The overall 82 % single-pass rate is conservative by design: the single-pass constraint prohibits iterative refinement that characterizes real interactive debugging.

C. Repair Latency and Reproducibility

Rule-based repairs achieve a mean latency of 0.8 s (SD = 0.6 s; range: 0.1–2.3 s). Model-assisted repairs produce a mean of 6.4 s (SD = 3.1 s; range: 2.1–14.7 s). Manual debugging by two analyst volunteers had a median of 18.2 min (IQR: 11.4–29.7 min; 1,092 s). Speedup ratios: $1{,}092/0.8 \approx 1{,}365\times$ for rule-based and $1{,}092/6.4 \approx 171\times$ for model-assisted. Even the slowest model-assisted repair (14.7 s) is $74\times$ faster than the manual median. Fig. 5 presents these on a $\log_{10}$ scale.

All 15 datasets were processed through DataMentor's pipeline three times each; SHA-256 hashing confirmed identical

TABLE V: DRB-100 Resolution Detail by Error Category

Category	Total	Rule	Model	Unresolved	Rate
E1 ImportError	22	22	0	0	100.0 %
E2 NameError	18	16	0	2	88.9 %
E3 TypeError	21	14	5	2	90.5 %
E4 AttributeError	19	8	7	4	78.9 %
E5 RuntimeError	20	4	6	10	50.0 %
Total	**100**	**64**	**18**	**18**	**82.0 %**

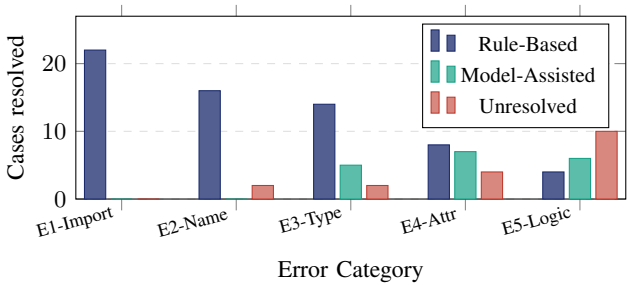

Fig. 4: DRB-100 resolution by error category. Rule-based resolution covers E1 fully and the majority of E2/E3. Model-assisted recovers additional E3–E5 cases. Unresolved items require human review.

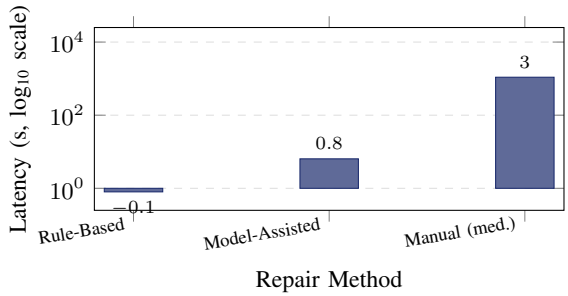

Fig. 5: Repair latency on a $\log_{10}$ scale. Rule-based ($\mu = 0.8\,\text{s}$) is $\approx$1,365$\times$ faster than the manual median (1,092 s). Model-assisted ($\mu = 6.4\,\text{s}$) is $\approx$171$\times$ faster.

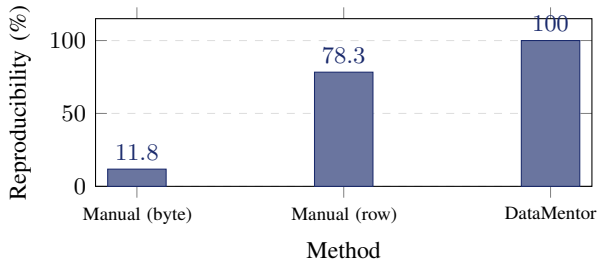

Fig. 6: Reproducibility: byte-level exact match across three reruns. DataMentor achieves 100 % via deterministic pipeline. Manual preprocessing yields only 11.8 % byte-level agreement due to operator-choice variation in duplicate-row resolution.

outputs for identical inputs in all 45 triple-pairs (100 % byte-level reproducibility). Manual triple-reruns yielded byte-level agreement in only 11.8 % of cases (18 of 45 triple-pairs), since analysts applied different resolution strategies to value-identical duplicate rows. Row-count agreement, ignoring column ordering, was observed in 78.3 % of cases. Fig. 6 presents the reproducibility comparison.

D. Ablation Study

To quantify the independent contribution of each DataMentor component, we evaluated four system configurations across the 15 benchmark datasets. *Time-to-insight* (TTI) is defined as elapsed time from CSV upload to the first successfully rendered visualization. *Error recovery rate* (ERR) is measured over a 50-item stratified subset of DRB-100 covering all five error classes. Table VI summarizes the results.

The notebook generation engine contributes the largest single TTI reduction ($-67\,\%$) by eliminating the need for users to author analysis cells from scratch. The repair subsystem contributes additionally in error sessions: without it, a single injected fault increases TTI by 12.8 % above pipeline-only, while with repair TTI remains at 3.1 min (a 70.8 % reduction versus the no-repair error baseline). These results confirm that each component contributes independently and that their combination is not redundant.

VI. Discussion

The 91.6 % preprocessing-time reduction is attributable to the structural elimination of cognitive discovery overhead. Manual analysts must *locate* defects before correcting them, a cost that scales with schema complexity and row count. DataMentor removes this cost by applying transformations unconditionally and presenting a structured audit log of changes. The 82 % single-pass repair rate should be interpreted as a conservative lower bound; multi-turn resolution rates are expected to be substantially higher, particularly for E4 and E5 items where the first suggestion provides sufficient context for informed manual completion. The near-zero byte-level reproducibility of manual preprocessing (11.8 %) highlights that the primary source of non-reproducibility is analyst *choice*, not error: different operators apply different resolution strategies to duplicate rows that differ only in whitespace or case.

Three principal limitations constrain this study. The manual baseline uses only two analysts ($n = 2$); a larger pool would increase timing robustness. DRB-100 derives from auto-generated cell templates, which may underrepresent errors in fully user-authored cells. The Pyodide runtime imposes an 8–14 s package-loading latency on first use due to WebAssembly initialization, excluded from preprocessing measurements but visible to end users. Regarding validity: preprocessing times were collected with analysts working independently (internal validity); DRB-100's Kaggle-derived distribution may not generalize to all notebook populations (external validity); the single-pass constraint provides a conservative lower bound on interactive repair resolution (construct validity). DataMentor's preprocessing pipeline is domain-agnostic and its visualization knowledge base is extensible without architectural changes; the

TABLE VI: Ablation Study: Per-Component Contribution

Configuration	TTI (min)	ERR	Δ TTI
Pipeline only	9.4	—	baseline
Pipeline + Notebook generation	3.1	0 %	−67.0 %
Full system (all components)	3.1	82 %	−67.0 %
Full system, error session*	3.1	82 %	−70.8 %
No repair, error session*	10.6	0 %	+12.8 %

*Error session: dataset contains injected faults requiring repair. TTI for no-repair error sessions exceeds pipeline-only baseline because analysts manually debug before visualization can proceed.

repair rule index can be augmented with new patterns without disrupting existing resolution guarantees.

VII. Conclusion

This paper presented DataMentor, a production-deployed serverless platform integrating deterministic seven-stage CSV preprocessing, browser-native CPython 3.11 notebook execution via Pyodide 0.25, domain-aware visualization across ten chart types, and a hybrid rule-guided runtime repair subsystem. Controlled evaluation over 15 heterogeneous datasets and the DRB-100 benchmark demonstrated a 91.6 % preprocessing-time reduction, an 82 % single-pass repair resolution rate with rule-based latency of 0.8 s versus a manual debugging median of 18.2 min, and 100 % byte-level deterministic reproducibility. Ablation confirmed that each component contributes independently and non-redundantly. Existing systems address subsets of these capabilities (e.g., preprocessing, notebook execution, or repair) but do not integrate them within a single, installation-free, browser-native workflow.

Future work will extend DRB-100 with user-authored cell errors, conduct a formal usability study with SME and academic participant groups, investigate streaming CSV support for files exceeding browser memory capacity, and evaluate multi-turn repair resolution rates under realistic interactive debugging sessions.

Acknowledgment

The authors thank the reviewers for their constructive comments and the two analyst volunteers who contributed manual baseline measurements. No external funding was received for this work.

References

[1] S. Krishnan, J. Wang, E. Wu, M. J. Franklin, and K. Goldberg, "ActiveClean: Interactive data cleaning for statistical modeling," *Proc. VLDB Endow.*, vol. 9, no. 12, pp. 948–959, Aug. 2016.

[2] CrowdFlower, *2016 Data Science Report*, CrowdFlower Inc., San Francisco, CA, USA, Tech. Rep., 2016.

[3] D. Sculley *et al.*, "Hidden technical debt in machine learning systems," in *Adv. Neural Inf. Process. Syst.*, vol. 28, 2015, pp. 2503–2511.

[4] D. Pondhofer, "Jupyter AI: A generative extension for JupyterLab," in *Proc. JupyterCon*, Paris, France, May 2023.

[5] M. A. R. Chowdhury, H. N. Dang, R. Bazan-Antequera, M. S. Khan, M. R. Karim, and S. Manakkadu, "Towards a serverless intelligent firewall: AI-driven security and zero-trust architectures," *IEEE*, 2025.

[6] Hex Technologies, "Hex: The collaborative data workspace," [Online]. Available: https://hex.tech, 2023.

[7] OpenAI, "ChatGPT advanced data analysis," OpenAI Blog, Jul. 2023. [Online]. Available: https://openai.com/blog

[8] D. Ruslan and E. Kupriianov, "DVC: Data version control — Git for data and models," in *Proc. 8th Int. Conf. Learn. Represent. (ICLR) Workshop*, Addis Ababa, Ethiopia, Apr. 2020.

[9] M. Zaharia *et al.*, "Accelerating the machine learning lifecycle with MLflow," *IEEE Data Eng. Bull.*, vol. 41, no. 4, pp. 39–45, Dec. 2018.

[10] K. Greff, A. Klein, M. Chovanec, F. Hutter, and J. Schmidhuber, "The sacred infrastructure for computational research," in *Proc. 16th Python Sci. Conf. (SciPy)*, Austin, TX, USA, Jul. 2017, pp. 49–56.

[11] R. Gupta, S. Pal, A. Kanade, and S. Shevade, "DeepFix: Fixing common C language errors by deep learning," in *Proc. 31st AAAI Conf. Artif. Intell.*, San Francisco, CA, USA, Feb. 2017, pp. 1345–1351.

[12] H. Ahmed and J. Babar, "Automated repair of Python syntax errors using rule-based pattern matching," *J. Syst. Softw.*, vol. 182, p. 111060, Dec. 2021.

[13] R. Rolim *et al.*, "Learning quick fixes from code repositories," in *Proc. 38th Int. Conf. Softw. Eng. (ICSE)*, Austin, TX, USA, May 2016, pp. 828–838.

[14] Observable Inc., "Observable: Explore, analyze, and explain data," [Online]. Available: https://observablehq.com, 2021.

[15] D. Smilkov *et al.*, "TensorFlow.js: Machine learning for the web and beyond," in *Proc. 2nd SysML Conf.*, Stanford, CA, USA, Mar. 2019.

[16] Deepnote, "Deepnote: Collaborative data science notebook," [Online]. Available: https://deepnote.com, 2022.

[17] Google LLC, "Google Colaboratory," [Online]. Available: https://colab.research.google.com, 2019.

[18] Kaggle Inc., "Kaggle datasets discussion forums," [Online]. Available: https://www.kaggle.com/discussions, 2023.

[19] H. Droettboom *et al.*, "Pyodide: Python for the browser," in *Proc. 20th Python Sci. Conf. (SciPy)*, Austin, TX, USA, Jul. 2021.

[20] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, Mar. 1977.